



Digital Design

Chapter 1: Introduction

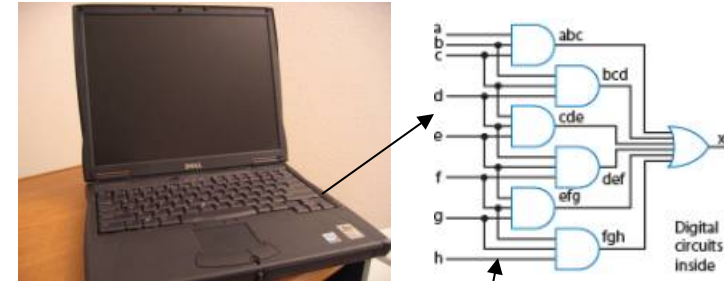
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Why Study Digital Design?

- Look “under the hood” of computers
 - Solid understanding --> confidence, insight, even better programmer when aware of hardware resource issues
- Electronic devices becoming digital
 - Enabled by shrinking and more capable chips
 - Enables:
 - Better devices: Better sound recorders, cameras, cars, cell phones, medical devices,...
 - New devices: Video games, smart phones, ...
 - Known as “embedded systems”
 - Thousands of new devices every year
 - Designers needed: Potential career direction



	Satellites		DVD players		Video recorders		Musical instruments
Portable music players	Cell phones	Cameras	TVs	???			

1995

1997

1999

2001

2003

2005

2007

Digital Design
Copyright © 2007
Frank Vahid

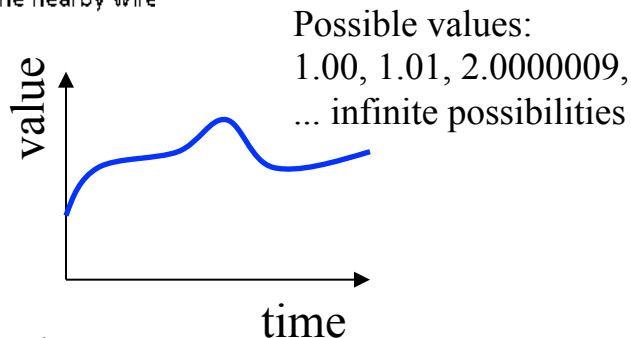
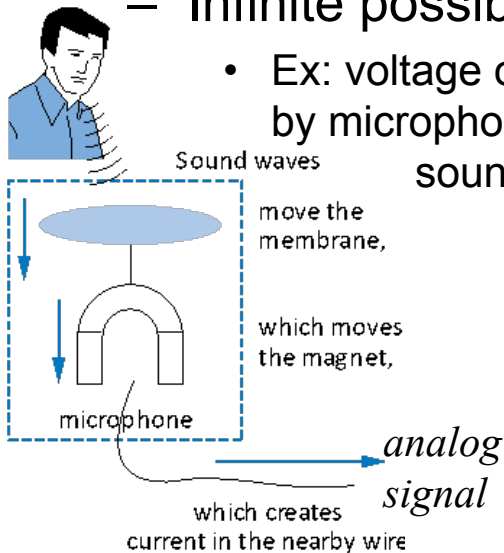
• Years shown above indicate when digital version began to *dominate*
– (Not the first year that a digital version appeared)

What Does “Digital” Mean?

- Analog signal (continuous signal)

- Infinite possible values

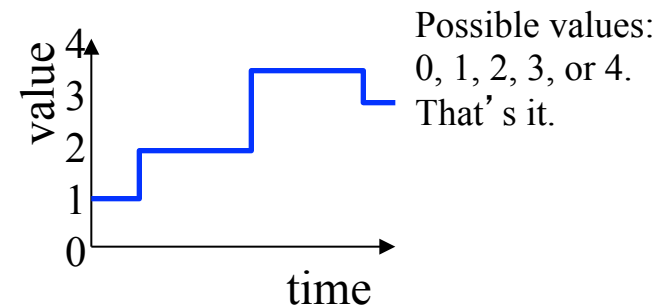
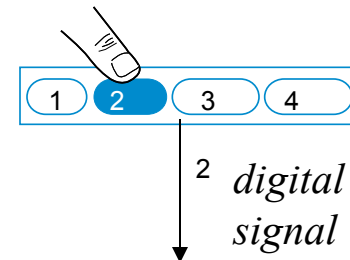
- Ex: voltage on a wire created by microphone, temperature, sound, image



- Digital signal (discrete signal)

- Finite possible values

- Ex: button pressed on a keypad, horn honked/not

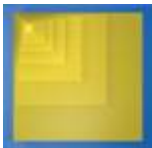


What Does “Digital” Mean?

The word ***digital*** comes from the same source as the words digit and digitus (the Latin word for finger), as fingers are used for discrete counting. (e.g. binary and decimal digits)

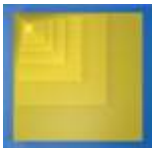
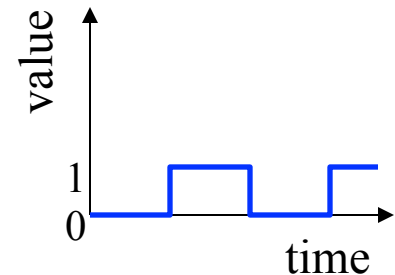
A digital system is a system that takes digital inputs (discrete signals) and produces digital outputs.

Although digital representations are discrete, the information represented can be either discrete, such as numbers and letters or continuous, such as sounds, and images.



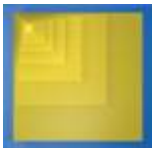
Digital Signals with Only Two Values: Binary

- **Binary** digital signal -- only *two* possible values
 - Typically represented as **0** and **1**
 - One **binary digit** is a **bit**
 - We'll only consider *binary* digital signals
 - Binary is popular because
 - Transistors, the basic digital electric component, operate using *two* voltages (more in Chpt. 2)
 - Storing/transmitting one of *two* values is easier than three or more (e.g., loud beep or quiet beep, reflection or no reflection)



Representing Information in Computers

- Electronic computers represent information as voltage levels.
- To make the computer hardware simple and reliable, computers represent information in *binary* form.
 - example: voltages greater than 3V are interpreted as representing one value (called “1”), voltages less than 2V are interpreted as representing another value (called “0”).
- In principle, could use more voltage levels.
 - example: 0 to .75V represents “0”, 1 to 1.75V represents “1”, 2 to 2.75V represents “2”, and so forth.
- In practice, this is rarely done.
 - requires more complex circuits
 - circuits are more susceptible to noise, hence less reliable



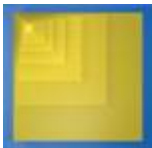
Why “Digital”?

World is mostly analog and therefore many applications were previously implemented with analog circuits. But this is changing.

- Humans often obtain advantages when converting analog signals to digital signals before processing that information. E.g. car horn. On or off

Advantages of Digital:

- Quality: Lack of deterioration in quality over time.
- Compression
- Reproducibility of results,
- Ease of design, Flexibility and functionality, Programmability, Speed, economy, steadily advancing technology



Example of Digitization Benefit: Quality

- Analog signal (e.g., audio) may lose quality (audio tapes)
 - Voltage levels not saved/copied/transmitted perfectly
- Digitized version enables near-perfect save/cpy/trn. (Compact Discs 6bil bits)
 - “Sample” voltage at particular rate, save sample using bit encoding
 - Voltage levels still not kept perfectly
 - But we can distinguish 0s from 1s

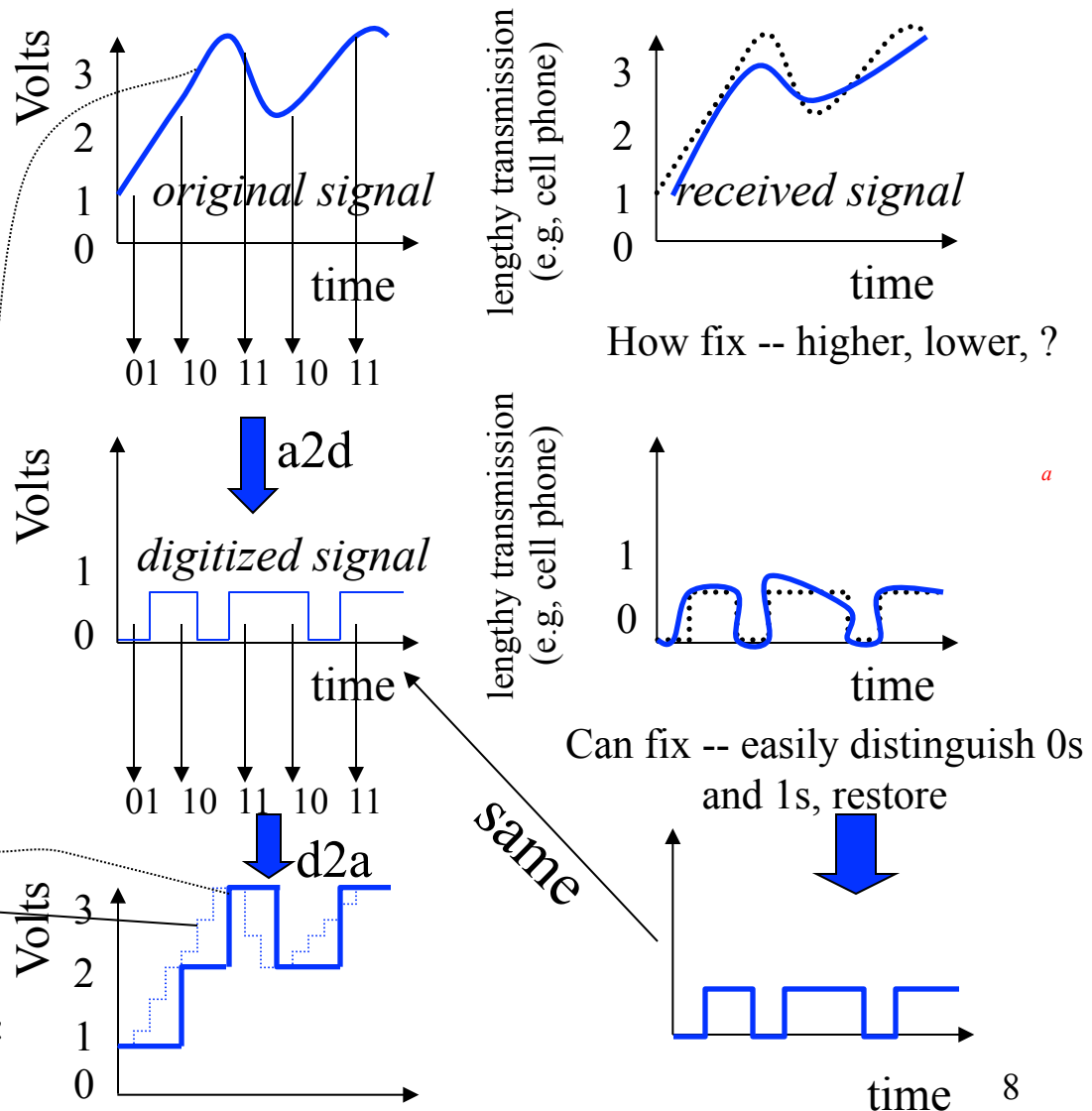
Let bit encoding be:

1 V: “01”
2 V: “10”
3 V: “11”

Digitized signal not perfect re-creation, but higher sampling rate and more bits per encoding brings closer.



Digital Design
Copyright © 2007
Frank Vahid



Digitized Audio: Compression Benefit

- Digitized audio can be compressed
 - e.g., MP3s
 - A CD can hold about 20 songs uncompressed, but about 200 compressed
- Compression also done on digitized pictures (jpeg), movies (mpeg), digital cell phones and more
- Digitization has many other benefits too

Example compression scheme:

00 --> 0000000000

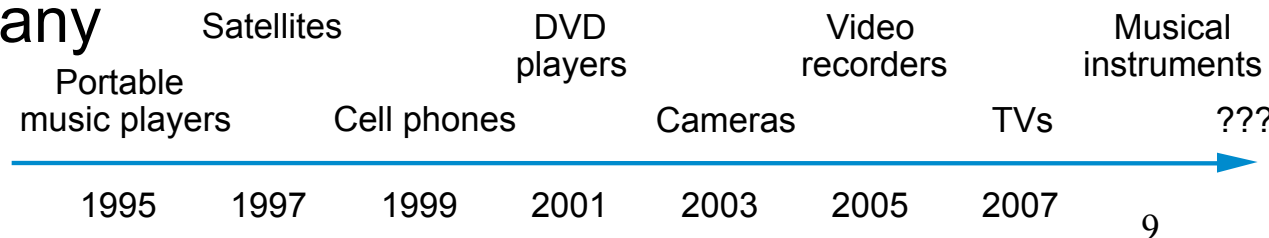
01 --> 1111111111

1X --> X

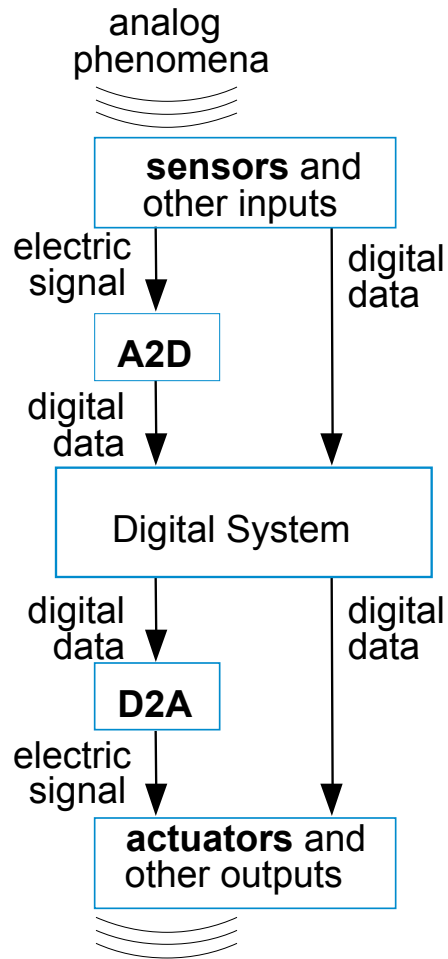
0000000000 0000000000 0000001111 1111111111
 ↘ ↘ ↘ ↘
 00 00 10000001111 01

If we sample hundreds or thousands of times a second, each sample 10 bits, we end up with huge numbers of bits.

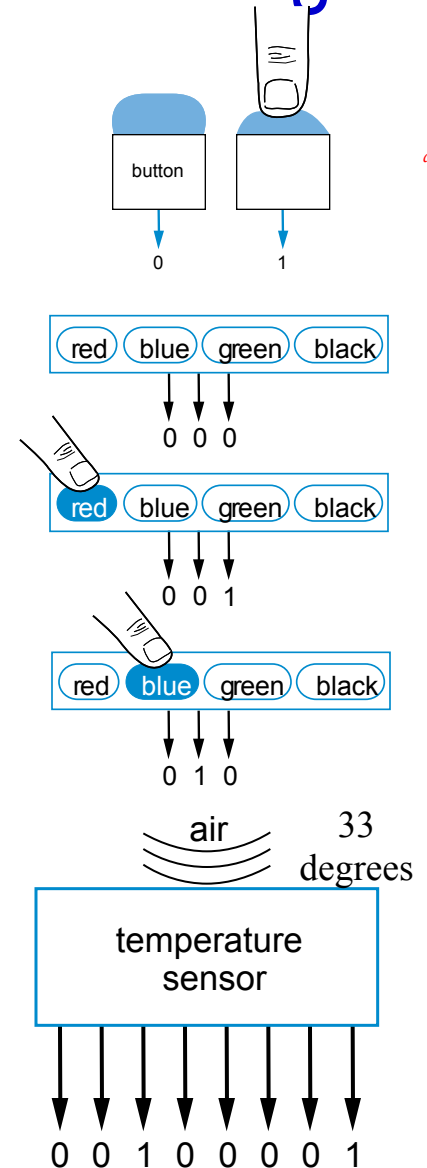
Conversion from analog to digital



How Do We Encode Data as Binary for Our Digital System?

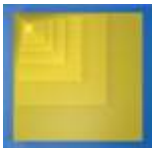


- Some inputs inherently binary
 - Button: not pressed (0), pressed (1)
- Some inputs inherently digital
 - Just need encoding in binary
 - e.g., multi-button input: encode red=001, blue=010, ...
- Some inputs analog
 - Need analog-to-digital conversion
 - As done in earlier slide -- sample and encode with bits



Example

- For example, automobiles digitize information about the engine temperature, car speed, fuel-level, air bags, brakes etc. so that an on chip computer can monitor and control the vehicle.



How to Encode Text: ASCII, Unicode

- ASCII: 7- bit encoding of each letter, number, or symbol
- Unicode: Increasingly popular 16-bit bit encoding
 - Encodes characters from various world languages

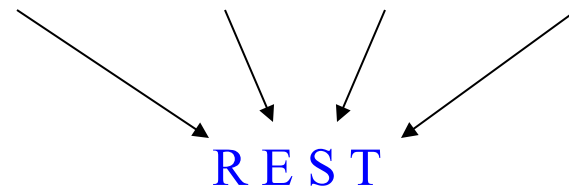
Symbol	Encoding
R	1010010
S	1010011
T	1010100
L	1001100
N	1001110
E	1000101
0	0110000
.	0101110
<tab>	0001001

Symbol	Encoding
r	1110010
s	1110011
t	1110100
l	1101100
n	1101110
e	1100101
9	0111001
!	0100001
<space>	0100000

Question:

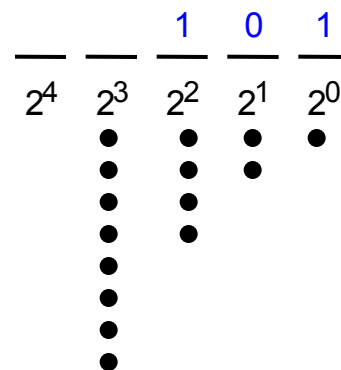
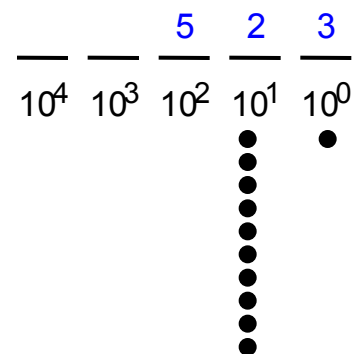
What does this ASCII bit sequence represent?

1010010 1000101 1010011 1010100



How to Encode Numbers: Binary Numbers

- Digital circuits perform arithmetic computations.
- Each position represents a quantity; symbol in position means how many of that quantity
 - Base ten (**decimal**)
 - Ten symbols: 0, 1, 2, ..., 8, and 9
 - More than 9 -- next position
 - So each position power of 10
 - Nothing special about base 10 -- used because we have 10 fingers
 - Base two (**binary**)
 - Two symbols: 0 and 1
 - More than 1 -- next position
 - So each position power of 2



Q: How much?

● + ● = ●
 ● ● ●
 ● ● ●
 ● ● ●
 4 + 1 = 5



How to Encode Numbers: Binary Numbers

- Working with binary numbers
 - In base ten, helps to know powers of 10
 - one, ten, hundred, thousand, ten thousand, ...
 - In base two, helps to know powers of 2
 - one, two, four, eight, sixteen, thirty two, sixty four, one hundred twenty eight
 - (Note: unlike base ten, we don't have common names, like "thousand," for each position in base ten -- so we use the base ten name)

2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
512	256	128	64	32	16	8	4	2	1

- Q: count up by powers of two

512 256 128 64 32 16 8 4 2 1 *a*



Converting from Decimal to Binary Numbers: Subtraction Method (Easy for Humans)

- Goal
 - Get the binary weights to add up to the decimal quantity
 - Work from left to right
 - (Right to left – may fill in 1s that shouldn't have been there – try it).

Desired decimal number: **12**

<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	
1						= 32
32	16	8	4	2	1	too much

<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	
0	1					= 16
32	16	8	4	2	1	too much

<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	
0	0	1				= 8
32	16	8	4	2	1	ok, keep going

<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	
0	0	1	1			= 8+4=12
32	16	8	4	2	1	DONE

<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	
0	0	1	1	0	0	answer
32	16	8	4	2	1	



Converting from Decimal to Binary Numbers: Subtraction Method (Easy for Humans)

- Subtraction method
 - To make the job easier (especially for big numbers), we can just subtract a selected binary weight from the (remaining) quantity
 - Then, we have a new remaining quantity, and we start again (from the present binary position)
 - Stop when remaining quantity is 0

Remaining quantity: 12

<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	
<u>1</u>						32 is too much
<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	

<u>0</u>	<u>1</u>					16 is too much
<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	

<u>0</u>	<u>0</u>	<u>1</u>				<u>12</u> - 8 = <u>4</u>
<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	

<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>			<u>4</u> - 4 = <u>0</u> DONE
<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	

<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>0</u>	answer
<u>32</u>	<u>16</u>	<u>8</u>	<u>4</u>	<u>2</u>	<u>1</u>	



Converting from Decimal to Binary Numbers: Subtraction Method Example

- Q: Convert the number “23” from decimal to binary

A: Remaining quantity

23

Binary Number

$$\begin{array}{r} 0 \\ 32 \end{array} \begin{array}{r} 0 \\ 16 \end{array} \begin{array}{r} 0 \\ 8 \end{array} \begin{array}{r} 0 \\ 4 \end{array} \begin{array}{r} 0 \\ 2 \end{array} \begin{array}{r} 0 \\ 1 \end{array}$$

$$\begin{array}{r} 23 \\ -16 \\ \hline 7 \end{array}$$

$$\begin{array}{r} 0 \\ 32 \end{array} \begin{array}{r} 1 \\ 16 \end{array} \begin{array}{r} 0 \\ 8 \end{array} \begin{array}{r} 0 \\ 4 \end{array} \begin{array}{r} 0 \\ 2 \end{array} \begin{array}{r} 0 \\ 1 \end{array}$$

$$\begin{array}{r} 7 \\ -4 \\ \hline 3 \end{array}$$

$$\begin{array}{r} 0 \\ 32 \end{array} \begin{array}{r} 1 \\ 16 \end{array} \begin{array}{r} 0 \\ 8 \end{array} \begin{array}{r} 1 \\ 4 \end{array} \begin{array}{r} 0 \\ 2 \end{array} \begin{array}{r} 0 \\ 1 \end{array}$$

8 is more than 7, can't use

$$\begin{array}{r} 4 \\ -2 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 0 \\ 32 \end{array} \begin{array}{r} 1 \\ 16 \end{array} \begin{array}{r} 0 \\ 8 \end{array} \begin{array}{r} 1 \\ 4 \end{array} \begin{array}{r} 1 \\ 2 \end{array} \begin{array}{r} 0 \\ 1 \end{array}$$

$$\begin{array}{r} 1 \\ -1 \\ \hline 0 \end{array}$$

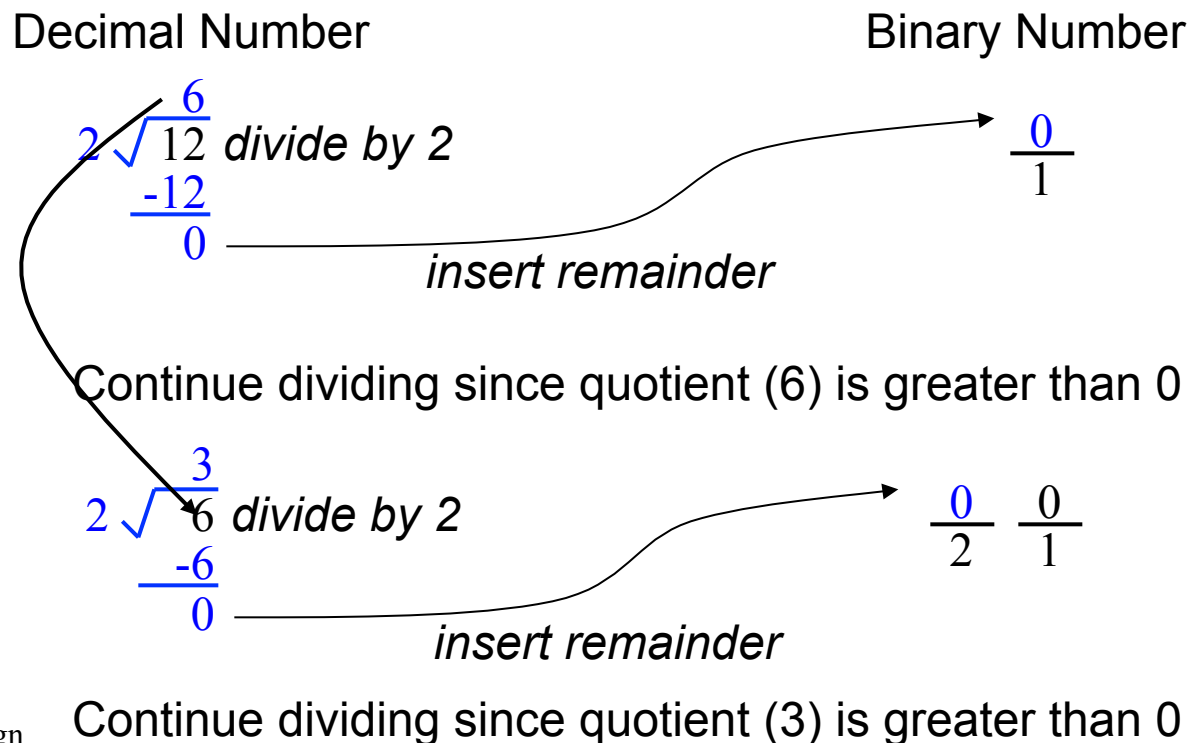
$$\begin{array}{r} 0 \\ 32 \end{array} \begin{array}{r} 1 \\ 16 \end{array} \begin{array}{r} 0 \\ 8 \end{array} \begin{array}{r} 1 \\ 4 \end{array} \begin{array}{r} 1 \\ 2 \end{array} \begin{array}{r} 1 \\ 1 \end{array}$$

Done! 23 in decimal is 10111 in binary.



Converting from Decimal to Binary Numbers: Division Method (Good for Computers)

- Divide decimal number by 2 and insert remainder into new binary number.
 - Continue dividing quotient by 2 until the quotient is 0.
- Example: Convert decimal number 12 to binary



Converting from Decimal to Binary Numbers: Division Method (Good for Computers)

- Example: Convert decimal number 12 to binary (continued)

Decimal Number

$$\begin{array}{r} 2 \overline{) 12} \\ \underline{-2} \\ 10 \end{array}$$

insert remainder

Binary Number

$$\frac{1}{4} \quad \frac{0}{2} \quad \frac{0}{1}$$

Continue dividing since quotient (1) is greater than 0

$$\begin{array}{r} 2 \overline{) 10} \\ \underline{-0} \\ 10 \end{array}$$

insert remainder

$$\frac{1}{8} \quad \frac{1}{4} \quad \frac{0}{2} \quad \frac{0}{1}$$

Since quotient is 0, we can conclude that 12 is 1100 in binary



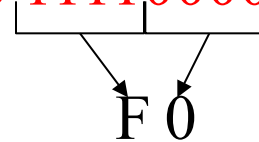
Base Sixteen: Another Base Sometimes Used by Digital Designers

—	—	8	A	F
16^4	16^3	16^2	16^1	16^0
		8	A	F
		↓	↓	↓
		1000	1010	1111

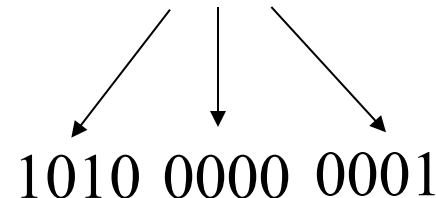
hex	binary	hex	binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

- Nice because each position represents four base two positions
 - Used as compact means to write binary numbers
- Known as **hexadecimal**, or just **hex**
- Often used to describe memory address space

Q: Write 11110000 in hex



Q: Convert hex A01 to binary



When converting from binary to hex

- Group bits into groups of four
- Start from the right



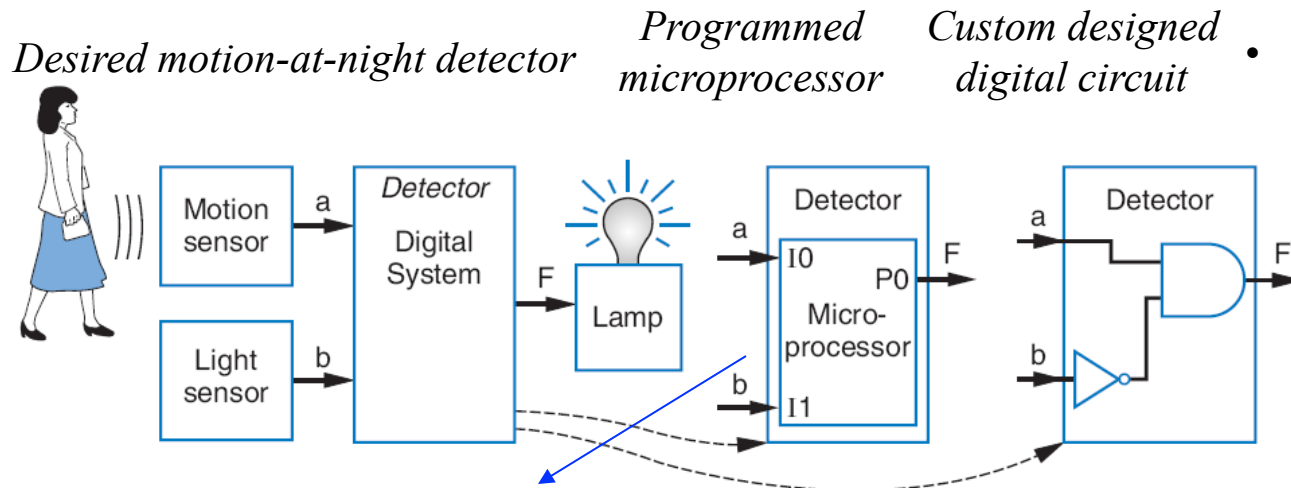
Bytes, Kilobytes, Megabytes, and More

- Byte: 8 bits 
- Common metric prefixes:
 - kilo (thousand, or 10^3), mega (million, or 10^6), giga (billion, or 10^9), and tera (trillion, or 10^{12}), e.g., kilobyte, or KByte
- BUT, metric prefixes also commonly used inaccurately
 - $2^{16} = 65536$ commonly written as “64 Kbyte”
 - $64 = 2^6$
 - kilo is 1024 not 1000 so **kilo = 2^{10}** so 64 KByte:= 2^{16} byte
 - Similarly, **mega= 2^{20}**
 - Typical when describing memory sizes
- Also watch out for “KB” for kilobyte vs. “Kb” for kilobit



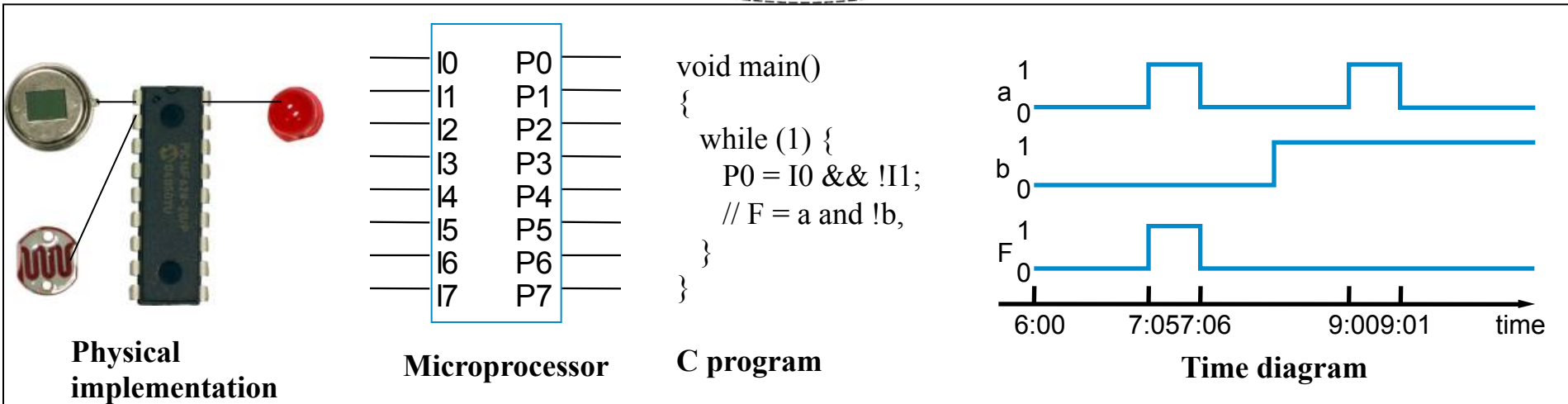
Implementing Digital Systems: Programming

Microprocessors Vs. Custom Digital Circuits



• Microprocessors a common choice to implement a digital system

- Easy to program
- Cheap (as low as \$1)
- Available now

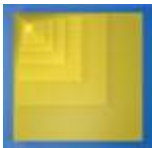


Digital Design: When Microprocessors Aren't Good Enough

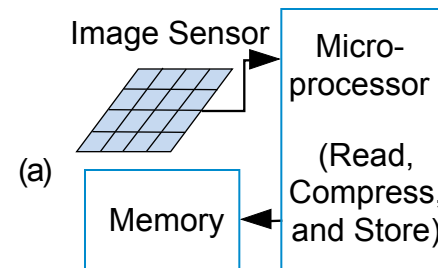
- With microprocessors so easy, cheap, and available, why design a digital circuit?
 - Microprocessor may be too slow; video compression
 - Or too big, power hungry, or costly; pentium for motion sensor

Sample digital camera task execution times (in seconds) on a microprocessor versus a digital circuit:

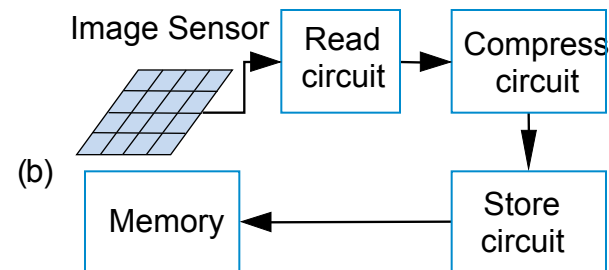
Task	Microprocessor	Custom Digital Circuit
Read	5	0.1
Compress	8	0.5
Store	1	0.8



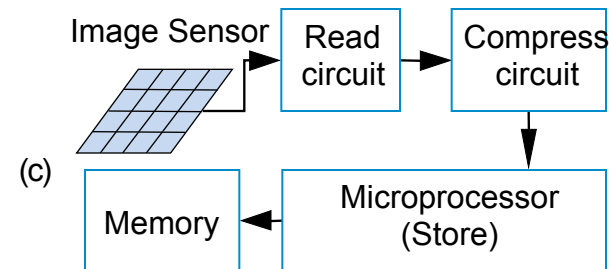
Q: How long for each implementation option?



$$5+8+1=14 \text{ sec}$$



$$.1+.5+.8=1.4 \text{ sec}$$



$$.1+.5+1=1.6 \text{ sec}$$

Good compromise

Chapter Summary

- Digital systems surround us
 - Inside computers
 - Inside huge variety of other electronic devices (embedded systems)
- Digital systems use 0s and 1s
 - Encoding analog signals to digital can provide many benefits
 - e.g., audio -- higher-quality storage/transmission, compression, etc.
 - Encoding integers as 0s and 1s: Binary numbers
- Microprocessors (themselves digital) can implement many digital systems easily and inexpensively
 - But often not good enough -- need custom digital circuits
- Next lecture, number systems from Wakerly 2.1-2.3

